

OMNI

Builder's Planning Book, Manual, and Roadmap

A large, high level handbook for designing a Windows first, government grade analytics platform with resilient offline operation, strong boundaries, and a Palantir like analyst workflow.

Prepared for the OMNI concept under Invincible, Inc.

This edition intentionally keeps every major idea family from the preserved brief visible, but any capability area that would require operationally sensitive implementation detail is handled only at a planning level.

How to read this edition

This is a planning book, not a coding sprint note. It explains what to build, how to break the work apart, how the app should look and feel, why each architectural move exists, and where some capability families are deliberately kept abstract. The tone is conversational on purpose, but the language is fully cleaned up so the document reads like a proper handbook.

Contents

- Part I. What you are really building
- Part II. The book lessons that should shape the app
- Part III. Product vision, users, language, and bounded contexts
- Part IV. The actual layout and appearance of the app
- Part V. Core platform architecture and three operating modes
- Part VI. Analytics workflow, evidence handling, and operator trust
- Part VII. Development roadmap, checks and balances, and team structure
- Part VIII. Capability families kept visible at a deliberately high level
- Appendix A. Preserved brief, verbatim
- Appendix B. Working glossary for the OMNI team

Part I. What you are really building

1. The plain English version

Let us start with the simplest possible explanation. You are not really building a giant map, a dashboard, a graph database, or an AI wrapper. You are building an operator's working environment. That matters, because when you name the project correctly, a lot of design decisions become easier.

The app should feel like a place where a user can take a messy stream of evidence, turn it into a clean operating picture, decide what matters, and move from observation to approved action without losing context. The software is not valuable because it collects a lot of data. It is valuable because it helps a human team understand what they are seeing, trace why they believe it, and coordinate what happens next.

That is why the center of gravity of OMNI is not the data store. It is the relationship between five things: the case, the entities involved, the evidence attached to those entities, the geospatial and time context around the evidence, and the workflow that lets a team move from uncertainty to a disciplined decision.

Why this framing matters

If you think you are building a data bucket, you will overinvest in ingestion and underinvest in human reasoning, interface clarity, approvals, traceability, and continuity. If you think you are building an analyst workbench, you are much more likely to put the domain model, the evidence chain, the workflow state, and the screen layout in the right order.

2. The product promise

The promise of OMNI is simple to say and hard to do well: one app, one operating picture, many sources, many workflows, and one clear path from raw inputs to usable action. In a healthy environment, the platform should use strong back end services, large scale storage, model serving, and shared indexes. In a damaged environment, the same operator experience should degrade gracefully onto a smaller node. In a continuity emergency, the same core thinking tools should still work on a single machine with a carefully limited but still useful subset of data and features.

So the product promise has three layers. First, it must be useful. Second, it must stay coherent as it grows. Third, it must fail gracefully rather than fail theatrically.

3. What this manual keeps and what it abstracts

You asked very clearly that major ideas not be removed. This edition follows that rule. The broad capability families from the preserved brief remain present throughout the handbook. However, where a capability family would require sensitive operational detail, exploit instructions, intrusive surveillance playbooks, or offensive implementation guidance, the book keeps that

family visible only as a strategic placeholder. In other words, the category remains on the map, but the dangerous step by step is not in this manual.

That is not just a safety choice. It is also a good design choice. Early planning documents should preserve scope and intent while refusing to pretend that every future module is ready for detailed build notes.

Part II. The book lessons that should shape the app

4. Clean Architecture lessons you should apply on purpose

The architecture book contributes the backbone. Its main lesson for OMNI is that policy should sit at the center and details should sit at the edge. That means the core of the app should describe cases, entities, evidence, time windows, geospatial relationships, approvals, alerts, and operator workflows, while things like the database engine, the AI provider, the map rendering toolkit, the message bus, and the desktop shell are treated as replaceable details.

In practical terms, that means your highest value code should be the domain and use case code. A use case like 'open a case', 'merge two candidate entities', 'score evidence confidence', 'promote a watch item into an alert', 'prepare an approval package', or 'replay the state of a case at time T' should not care whether the UI is WinUI, whether the cache is Redis, or whether a model comes from a cloud service or a local model runtime.

This is the dependency rule translated into OMNI language. The inner layers are allowed to know about business meaning. The outer layers are not allowed to pull the inner layers into framework shaped compromises.

- Treat the desktop shell as a shell, not as the heart of the system.
- Treat the database as a persistence mechanism, not as your domain model.
- Treat AI as an advisor and transformer, not as the owner of truth.
- Treat the map as one view onto the model, not as the model itself.
- Treat adapters as translation edges, not as places where business rules quietly accumulate.

Why this matters

The moment you let the map library, the ORM, the message schema, or the AI output format dictate your domain objects, the platform becomes brittle. You will feel fast for a few weeks and then slow for years.

The same book is also useful for another reason. It keeps repeating the idea that frameworks are tools and details are details. That is exactly the right attitude here. The OMNI team should choose frameworks that help, but it should never let a framework become the shape of the product. If you later change from one graph store to another, or from one desktop host to another, the inner model should not need a spiritual crisis.

5. Domain Driven Design lessons you should use to stop the app from becoming mush

The domain design book is what prevents this project from turning into a giant pile of widgets connected to a giant pile of tables. Its most useful gift is the idea that language, model, and implementation should coevolve together. If the team cannot describe the work clearly, the software will not stay clear.

That means you need a real ubiquitous language, and it needs to be enforced across product, design, development, and operations. Decide what a case is. Decide what an entity is. Decide what an observation is. Decide what an evidence packet is. Decide what a subject profile is. Decide what an alert is. Decide what a workflow state is. Decide what a confidence rating means. Decide what a watchlist item is. Then make the software, the field labels, the design files, the API schemas, and the team conversations all use the same language.

The book is also right that the model and the implementation cannot be cleanly separated. You do not get a real domain model by drawing boxes in meetings and then tossing them over a wall. The team members who shape the model need to touch real code, and the people who write code need contact with the domain problem.

How to use this on OMNI

Whenever the team argues about a screen, stop and ask what domain object the screen is really exposing. If no one can answer that cleanly, the UI is probably compensating for an unclear model.

Bounded contexts matter just as much. A case management model, a sensor ingestion model, a user identity model, and an update delivery model should not all live in one conceptual soup. They can relate, but they should not collapse into each other. The fewer accidental meanings a model has to carry, the easier it is to evolve.

6. Effective Java lessons that matter even when parts of the stack are not Java

Even if OMNI ends up using a mixed stack with C#, C++, Python, and some data tooling around the edges, the discipline from the Java book still matters. The book is full of rules that are really about interface quality, object safety, maintainability, and long lived code.

Three rules matter immediately. First, prefer builders when the object has many optional attributes. OMNI is full of objects like search requests, filter sets, evidence queries, workspace layouts, alert definitions, and export packages. These should not be built with unreadable telescoping constructors or giant mutable bags. Second, prefer dependency injection over hardwiring resources. Every part of the platform that talks to storage, messaging, model serving, or external APIs should get those resources through well defined interfaces. Third, minimize mutability. In a system with audit trails, collaboration, replay, and concurrency, immutable request models and snapshot objects are worth a lot.

- Use interfaces at boundaries so implementations can change without ripping up call sites.
- Document API behavior like a product surface, because internal APIs eventually become permanent.
- Design for failure atomicity, because partial writes and half finished transitions create false confidence.
- Be strict with concurrency, because shared mutable state quietly destroys trust in a platform like this.

- Prefer composition over inheritance, because capability modules in a big app always age better as compositions.

7. Structured analytic technique lessons that should become product features

The two tradecraft primers are useful because they describe disciplined ways to reason under uncertainty. Even if you remove the agency specific context, the core lessons are excellent for product design. The app should not merely store evidence. It should help users frame the issue, check the source, check the quality of information, check relevance, generate alternative hypotheses, compare them, monitor indicators, and test assumptions over time.

This suggests a product rule: any important conclusion in OMNI should be able to show the issue statement, the evidence set, the source confidence judgment, the quality judgment, the relevance judgment, the alternative explanations considered, the indicators being watched, and the last review date.

Product implication

If the user can make a strong claim without showing why the evidence was considered relevant, what source quality was assigned, what alternatives were rejected, and what indicators might change the picture later, the app is helping people feel certain without helping them be disciplined.

8. Operational environment modeling lessons you can borrow without dragging the whole military workflow into the product

The battlespace preparation manual is useful because it is systematic. Strip away the branch specific language and you get a strong generic workflow for understanding an operational environment. First, define the environment and its boundaries. Second, describe how the environment affects operations. Third, evaluate the actors that matter. Fourth, determine the plausible courses of action within that environment.

For OMNI, that translates into a reusable analysis pattern. Define the area, timeframe, subject set, and legal or policy scope. Describe environmental factors that matter, such as terrain, infrastructure, weather, events, population patterns, network conditions, or operational constraints. Evaluate the relevant actors. Then generate plausible scenarios, workflow branches, or likely developments.

This is especially useful for a map first product. It keeps the map from being decorative. The map becomes one part of a broader operational environment model.

9. Targeting doctrine, reframed as objective alignment rather than engagement mechanics

The joint targeting manual is not something this handbook uses for action mechanics. What it does use is the idea that actions should be tied logically to objectives, prioritized systematically,

matched to available capabilities, and assessed afterward. That is a good planning loop for almost any serious decision support platform.

So in OMNI, the safe translation is this: objectives drive action packages; action packages are matched to authorized capabilities; execution choices are assessed against the intended effect; and weak or ambiguous matches are visible before anyone approves anything. This is not a license for hidden automation. It is a design rule that says action planning should be explicit, prioritized, and reviewable.

10. The SIGINT handbook lesson that most teams forget

The small SIGINT handbook article contributes one surprisingly important idea. Users need a plain language guide to how the system works, how outputs are produced, what flags mean, how terminology is used, and what the limits are. In other words, an advanced platform still needs user education built into the product.

That means OMNI should ship with a real operator handbook, inline field help, glossary panels, traceability drawers, and a 'why am I seeing this' explanation surface. A platform that looks powerful but cannot explain itself will be feared, misused, or ignored.

Part III. Product vision, users, language, and bounded contexts

11. Product vision

OMNI should feel like a command center, but the useful kind, not the movie kind. The user should always know what case they are in, what they are looking at, what time window is active, what filters are active, what the confidence level is, and what actions are available from that exact state.

The visual and functional inspiration is the high density operational software pattern people associate with Palantir like products: linked objects, a serious map, time based context, evidence traceability, saved workspaces, analyst tools that stay close to the data, and action paths that do not require leaving the operating picture.

But do not build a costume. Build the working mechanics underneath the look. The reason those interfaces feel powerful is not just the color scheme. It is the way every click tends to preserve context.

12. User roles

Core roles in the OMNI ecosystem

Role	Primary job	What the product must optimize for
Watch officer	Maintains situational awareness, triages alerts, escalates cases	Fast open, pin, compare, assign, and brief workflow
Analyst	Builds cases, evaluates evidence, develops hypotheses, writes conclusions	Deep evidence tools, explanation views, matrices, timelines
Case lead	Owens case state, quality, tasks, and approvals	Workflow state control, review queues, sign off surfaces
Operations planner	Turns a situation picture into authorized plans or coordination packages	Scenario tools, action package drafts, resource visibility
Commander or executive reviewer	Needs rapid comprehension, confidence cues, and decision packages	Condensed dashboards, summary narrative, drill down on demand
Data steward	Maintains source catalog, schema quality, lineage, retention, and policy tags	Registry, lineage, validation, and lifecycle tools
Platform administrator	Manages tenants, permissions, deployments, health, and recovery	Admin console, observability, update, rollback, continuity controls

Field node operator	Runs the platform in constrained or damaged environments	Local health, sync, storage pressure, model status, break glass controls
Legal or policy reviewer	Reviews actions, access levels, retention, and restricted workflows	Approval checkpoints, audit trails, policy explanations

13. Ubiquitous language for the OMNI team

Use these terms everywhere, in product copy, tickets, schemas, and code comments

Term	Meaning
Case	The container for a specific problem, mission, inquiry, or ongoing watch effort.
Entity	A person, organization, device, location, vehicle, network asset, or other thing with continuity through time.
Observation	A single reported or detected data point before broader evaluation.
Evidence packet	A grouped unit of observations and metadata that can be cited, reviewed, and scored.
Source profile	The structured description of where information came from and how much trust to assign.
Finding	A bounded conclusion supported by evidence.
Hypothesis	A plausible explanation being actively tested, not merely guessed.
Indicator	An observable sign that would support, weaken, or alter a hypothesis or scenario.
Alert	A rule based or analyst promoted item that needs attention within a time window.
Watchlist	A maintained set of entities, locations, topics, or patterns being monitored.
Action package	A prepared, reviewable bundle of recommended next steps, rationale, and approvals.
Workspace	A saved arrangement of screens, filters, and context for a specific job.
Snapshot	A frozen state of the case or platform that can be revisited later.

Continuity mode	A degraded but still functional operating posture when normal infrastructure is unavailable.
-----------------	--

14. The bounded context map

This is where the domain design book earns its keep. OMNI should be treated as one product composed of several bounded contexts, not one giant all knowing model. When teams fail to do this, everything becomes an 'object', every table joins to every other table, every screen starts needing exceptions, and nobody can explain ownership.

Primary bounded contexts

Bounded context	Owns	Boundary rule
Identity and Access	Users, roles, claims, offline access packets, break glass behavior	Nothing outside this context should invent its own permission logic
Case and Tasking	Cases, tasks, assignments, review states, action packages	Owns workflow, does not own raw ingestion details
Source Registry	Source catalog, lineage, provenance, validation rules, trust metadata	Owns source metadata, not conclusions
Evidence Graph	Entities, links, observations, evidence packets, merges, splits	Owns the working intelligence graph
Geo Temporal Workspace	Maps, tracks, timelines, zones, saved views	Owns location and time interaction patterns
Alerting and Monitoring	Rules, thresholds, subscriptions, watchlists, triage queues	Owns signal to attention conversion
Analytics and Hypotheses	Matrices, scenario tools, scoring helpers, indicator sets	Owns reasoning support, not source storage
Policy and Audit	Approvals, retention, redaction states, explanations, event logs	Owns compliance surfaces and audit truth
Continuity and Sync	Local nodes, cache packs, sync queues, conflict rules	Owns damaged environment operation
Update and Release Control	Install checks, snapshots, rollback, channels, diagnostics	Owns safe software change mechanics

The boundary rule in everyday language

If a team says, 'we will just reach into that other module and grab its tables directly', that is usually

the start of future pain. Make them go through a clean contract instead.

15. Domain objects, value objects, services, and aggregates

The OMNI model should distinguish between things with continuity and things that merely describe state. A case is an entity. An evidence packet is an entity. A subject profile is usually an entity. A confidence rating is a value object. A time window is a value object. A geographic region selection is a value object. A source profile might be an entity or aggregate root depending on how much lifecycle it carries. An alert rule is an entity. A display color palette is definitely not a domain concept at all, it is a presentation concern.

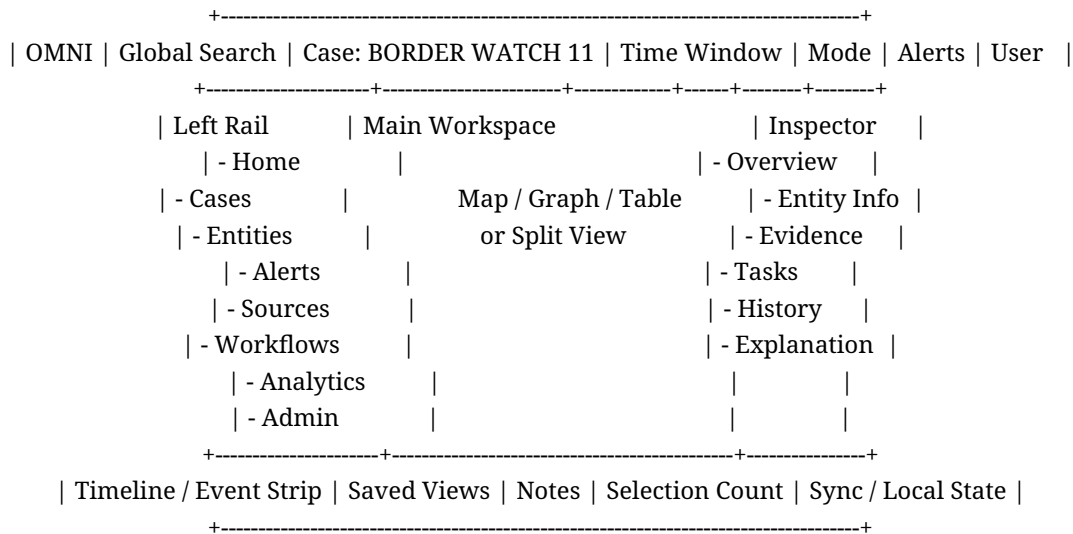
Likewise, some important operations are better modeled as services rather than stuffed awkwardly into an entity. Examples include entity resolution, evidence deduplication, confidence scoring, indicator evaluation, action package assembly, and workspace snapshot export.

- Aggregate roots should be few and intentional.
- Entities should guard their invariants instead of relying on the UI to behave.
- Value objects should be immutable whenever possible.
- Repositories should return domain shaped objects, not framework shaped record piles.
- Factories should create new conceptual objects, while repositories reconstitute existing ones.

Part IV. The actual layout and appearance of the app

16. The app shell

If you want the app to feel serious immediately, the shell has to be right. The shell is the discipline layer of the interface. It tells the user what is global, what is local, what is selected, what is filtered, and what can be acted on. A weak shell makes even powerful features feel random.



That should be the default mental map of the product. A strong top bar, a narrow but dependable left rail, a very large center workspace, a right side inspector, and a thin but useful bottom strip for time, notes, or system state.

17. The top bar, button by button

Top bar controls

Control	What it does	Why it belongs here
OMNI logo and home	Returns to the command overview without losing unsaved draft notes	The user needs a safe anchor
Global search	Searches entities, cases, locations, sources, and saved workspaces from one field	The best enterprise tools let users start from a noun or event and land somewhere useful
Current context chip	Shows current case or view and lets the user switch without context confusion	Analysts constantly forget which case they are in if the system does not help them
Time window control	Sets the active temporal filter for the entire workspace or selected panes	Time is one of the main organizing axes of this product

Operating mode indicator	Shows Cloud, Node, or Continuity mode with health and restrictions	The system must never hide degraded state
Notification and alert bell	Shows triage items, failed jobs, approval requests, and update notices	Users need one inbox for operational attention
Save workspace	Captures current pane arrangement, filters, selected entities, and map frame	Analysts repeat views more often than teams realize
Compare mode toggle	Splits the main workspace for side by side comparisons	Comparison is central to reasoning under uncertainty
User and permissions menu	Shows role, current tenant, session trust, and emergency options	Trust state must be visible at all times

18. The left rail

The left rail should be stable. It is not the place for cleverness. It is the place for dependable navigation.

- Home. The operational overview and personal start page.
- Cases. All active, paused, archived, and assigned cases.
- Entities. Entity registry and identity resolution workspace.
- Alerts. Triage queues, watchlists, subscriptions, and thresholds.
- Sources. Source registry, ingestion health, provenance, and schema quality.
- Workflows. Tasks, approvals, action packages, and review queues.
- Analytics. Hypothesis matrices, indicator boards, scenario tools, and evidence scoring.
- Admin. Policy, roles, diagnostics, updates, audit, and continuity controls.

19. The main workspace

The center of the screen should never be a dead dashboard. It should always be the place where real work happens. That workspace needs to support several modes, but all of them should preserve the same interaction grammar. Selection on the left should light up the center. Selection in the center should populate the inspector on the right. Time changes should ripple cleanly. Filters should be obvious and reversible.

Main workspace modes

Workspace mode	Best used for
Map mode	For geospatial work, route overlays, zones, clusters, and track playback
Graph mode	For relationship exploration between entities, evidence packets, and organizations

Table mode	For dense review of records, events, source attributes, and rule results
Timeline mode	For event ordering, bursts, long gaps, and synchronized evidence review
Split mode	For map plus inspector, graph plus timeline, or table plus map combinations
Brief mode	For executive review, summary cards, confidence, and drill down on demand

20. The right inspector

The right inspector is where focus becomes detail. It should open quickly, stay readable, and never bury the basics. The first panel for any selected object should answer five questions in order: what is this, why is it in view, how current is the information, how trustworthy is the underlying evidence, and what can I do next.

- Overview tab, plain language summary, tags, status, owners, last updated time.
- Evidence tab, citations, provenance, attachments, confidence, contradictions, and source notes.
- Relationships tab, nearby linked entities, parent or child context, cluster membership.
- History tab, state changes, merges, splits, approvals, and edits.
- Tasks tab, open actions, assigned reviewers, deadlines, and handoff notes.
- Explanation tab, why this result exists, what rules or models contributed, what changed recently.

21. The bottom strip

A lot of products waste the bottom strip. OMNI should use it for things that help the user stay oriented without opening extra panels.

- Timeline scrubber for current selection.
- Selection count and type summary.
- Local node and sync queue state.
- Short notes or scratchpad surface.
- Background task progress, such as exports, reindex, or model jobs.

22. Screen by screen layout guide

Now let us get concrete. Below is the actual screen plan I would hand to a product designer and a platform lead before wireframing starts in earnest.

Primary screens

Screen	Purpose	Primary contents
Command Overview	Default landing page after login	Operational summary cards, map mini view, high priority alerts, case queues, system health, saved workspaces
Case Workspace	The main collaborative working screen	Case header, objective panel, entity roster, evidence stream, hypotheses, tasks, approvals, timeline
Entity Profile	Everything known about one entity	Identity summary, aliases, confidence, related cases, linked evidence, map trail, watch status
Map Workspace	Geospatial and time coupled work	Layer controls, filters, route playback, region compare, event density, selected entity tracks
Evidence Explorer	Source first work	Source registry, packet review, confidence scoring, contradictions, attachments, lineage
Alert Triage	Fast attention management	Queues by severity and age, quick compare, escalation path, ownership, snooze and recheck
Analytics Studio	Reasoning support tools	Assumptions check, issue framing, matrix builder, indicators board, scenario canvases
Workflow and Approval Center	Where action packages move	Draft, review, approve, reject, route, annotate, audit
Admin and Continuity Console	System trust and health	Roles, policies, retention, local node status, update channel, rollback, diagnostics
Tray Utility	Quick operator controls outside the main window	Open app, check updates, view sync state, rollback, diagnostics, local mode status

23. Command Overview screen

Think of this screen as the answer to the question, 'What needs my attention in the next two minutes?' Not ten minutes, and not a deep investigation. Two minutes.

The top third should have six cards: open critical alerts, cases awaiting review, cases assigned to me, system health, local node health, and watchlist changes in the active time window. The center should hold a map mini view with the currently selected region and any urgent clusters. The right side of the center should show the active queue. The bottom should show recently used workspaces and recent case changes.

Design rule

The overview screen should never become a landfill of charts. Every panel should answer a real operator question, and every panel should open into a deeper workspace with one click.

24. Case Workspace

This is the soul of the product. If OMNI succeeds anywhere, it succeeds here.

The case header should show case title, objective, owner, current stage, confidence posture, classification or handling marker if applicable, last major update, and the current time window. Directly below that, use a three column working body.

+-----+					
Case Header: Name	Owner	Stage	Confidence	Last Update	Time Window
+-----+					
Left Column	Center Column		Right Column		
- Objective summary	- Evidence stream		- Current tasks		
- Entity roster	- Hypotheses panel		- Reviews		
- Key indicators	- Timeline / map / graph tab		- Approvals		
- Watch items	- Notes and draft findings		- Audit summary		
+-----+					

The left column keeps the nouns in view. The center column holds the active reasoning and evidence work. The right column keeps the workflow state visible so the case does not drift into a black box of endless analysis.

25. Entity Profile screen

The entity profile should feel like a durable dossier, but one that admits uncertainty. Do not present the entity as a monolith if the identity resolution is still weak. Show confidence, competing attributes, source diversity, and unresolved contradictions.

A strong entity profile contains: primary name, alternate names, category, summary, confidence bar, linked cases, recent observations, time activity chart, map trail, known relationships, watchlist status, and all evidence packets that contribute to the current profile view.

26. Map Workspace

The map is not decoration. It is a first class analysis surface. That means layers, filters, and playback all need to feel serious.

Map workspace building blocks

Map element	Purpose
Layer rail	Basemap, zones, entities, events, heat layers, routes, infrastructure, notes
Selection chip row	All active selections with quick clear and pin actions
Playback controls	Start, stop, step, speed, jump to event, jump to gap
Temporal histogram	Distribution of events by time, linked to the map frame
Region compare	Side by side panes for before and after, area A and area B, or subject A and subject B
Explanation drawer	Why this layer is shown, where it came from, and what filters affect it

27. Evidence Explorer

A serious platform cannot let evidence become a blurred background texture. The evidence explorer is where the platform proves that it respects provenance.

Organize this screen around packet review. The default list should show source family, packet title, ingest time, event time, reliability flag, quality flag, relevance flag, contradictions present, and case links. Opening a packet should reveal raw content, normalized content, metadata, derived entities, attached map coordinates, linked observations, and review history.

28. Alert Triage

The triage screen is all about speed without sloppiness. The layout should allow a reviewer to compare alerts, open source backing quickly, assign ownership, suppress false positives with a reason, and escalate a real alert into a case or an existing workflow.

Important rule: never let triage become model worship. Every alert needs a visible human readable reason. 'Model score 0.92' is not a reason. 'Unusual event burst near watched region combined with cross source corroboration' is closer to a reason.

29. Analytics Studio

This is where the lessons from the analytic primers become real product features. The studio should feel like a toolkit shelf, not a wizard that takes over the case.

Analytics studio toolset

Tool	What it should do
Issue framing board	Restate the question, broaden, narrow, flip perspective, identify hidden assumptions
Source check worksheet	Reliability, access, motivation, translation quality, collection limits, confidence notes
Quality check worksheet	Completeness, corroboration, ambiguity, contradictions, missing context
Relevance board	Why each evidence item matters to the actual question
Hypothesis matrix	Alternative explanations with supporting and conflicting evidence
Indicators board	Observable signposts that would strengthen or weaken a line of thinking
Scenario canvas	Alternative futures and branch conditions
Review package builder	Turns the studio output into a clean narrative summary for the case

Part V. Core platform architecture and three operating modes

30. The technical stack in plain language

Here is the stack I would plan toward. Use a shared cross platform core for domain, use cases, sync, storage contracts, and policy logic. Use a Windows first operator shell for the primary desktop experience. Use native performance modules only where the work actually demands them. Keep model serving behind adapters. Keep every external dependency outside the core.

Recommended stack shape

Layer	Likely technology shape	Reason
Desktop shell	C# with WinUI or WPF for the richest Windows operator client	Best productivity for a serious Windows first app
Portable operator core	.NET core libraries plus selected native modules	Lets the same logic run on Windows and Linux nodes
Performance modules	C++ for heavy parsing, spatial processing, or low level signal transforms that are legally authorized and already defined elsewhere	Use native code only where profiling proves it is worth it
Model and data science services	Separate services or local runtimes behind interfaces	Prevents AI tooling from owning the application shape
Storage	Mix of relational store, object storage, search index, and graph projection as needed	No single storage model should be forced to do everything
Messaging and jobs	Queue backed background processing	Ingestion, enrichment, scoring, export, sync, and diagnostics should not block the UI
Observability	Central logs, metrics, traces, and local diagnostic bundles	Enterprise software rots fast without visibility

31. Clean Architecture translated into a repo layout

```

/omni
/apps
/omni-desktop-windows
/omni-desktop-linux
/omni-tray
/omni-admin-console
/core
/domain

```

```

/usecases
/contracts
/policy
/sync
/adapters
/storage-sql
/storage-graph
/search
/maps
/models
/messaging
/auth
/modules
/evidence-graph
/case-workflow
/alerts
/analytics-studio
/geo-temporal
/update-rollbacks
/ops
/deploy
/diagnostics
/test-data
/continuity-node

```

This layout is not sacred, but the separation principle is. The inner folders should define meaning and behavior. The outer folders should implement integration and delivery.

32. Three operating modes

Your brief explicitly asks for three scenarios. So the architecture should be designed around three operating modes from day one, not added later as a marketing slide.

Required operating modes

Operating mode	Meaning	What should still work
Mode 1. Full networked operation	Normal server backed mode with distributed processing and shared services	Complete feature set, shared indexes, full collaboration, heavy ingestion, stronger model serving
Mode 2. Single node severe outage	One laptop or tower acts as the local backbone for a small team	Priority ingestion only, smaller local stores, queued sync, local model serving, constrained collaboration
Mode 3. Continuity emergency mode	One machine with only local resources and approved public or already held data	Local case work, evidence review, maps, timelines, hypotheses, notes, watchlists, exports, deferred sync

The real trick is not merely making each mode exist. The real trick is making them feel like the same product. Same nouns. Same workflows. Same screen layout. Same case object. Same evidence packet idea. Same approval package idea. Different capacity, not different product identity.

33. Data flow from source to operator

1. Register the source. Give every source family a source profile, trust metadata, ownership, retention, and schema rules.
2. Land the raw input. Do not destroy the original representation. Store it or its legally approved reference.
3. Normalize the data. Convert to stable internal packet shapes, but preserve a link to the raw form.
4. Enrich the packet. Extract entities, times, locations, tags, relationships, and quality markers.
5. Project into read models. Build graph views, map views, table indexes, timeline indexes, and alert indexes.
6. Expose the packet to review. Let humans score source quality, relevance, contradictions, and confidence.
7. Link the packet into cases and watchlists. Evidence becomes part of human work, not just machine storage.
8. Record every meaningful change in the audit trail. The system should be able to explain what changed and why.

34. The evidence graph

The evidence graph is the heart of the internal model. It should not be a magical graph database story. It should be a disciplined graph projection over a careful domain. At minimum, think in terms of entities, observations, evidence packets, relationships, locations, time intervals, cases, alerts, and action packages.

A useful rule is this: no edge should exist without a reason category, confidence marker, provenance path, and timestamps. If your graph can say that two things are linked but cannot say why, when, and based on what, the graph is theatrics.

35. Identity resolution

Identity resolution should be treated as an explicit service, not a side effect. Every candidate merge should carry confidence, rationale, and reversibility. That is especially important in serious environments. The product should support at least four states: clearly same, likely same, unresolved, and likely distinct. Do not pretend unresolved is a bug. Sometimes unresolved is the honest state.

36. AI in the platform

AI should help with transformation and suggestion, not replace accountability. The safe and durable places to use AI in OMNI are summarization, translation into normalized internal forms, clustering suggestions, entity resolution suggestions, explanation drafting, anomaly candidate ranking, and scenario draft generation.

- Every AI output should be marked as machine generated until reviewed.
- Every AI driven recommendation should show source packet references where possible.
- No AI output should silently mutate core truth records.
- Models should be hot swappable behind interfaces.
- Continuity mode should support reduced capability local models or no model fallback paths.

37. Security, trust, and policy architecture

Because the app is meant for serious use, trust and policy are not side concerns. They are part of the product grammar. Every case, entity view, evidence packet, action package, export, and update should be governed by visible policy and invisible enforcement.

Policy layers

Policy area	What it governs
Authentication	User identity, session trust, device trust, offline fallback
Authorization	Role, tenant, case, capability, data scope, and action level permissions
Audit	Every important read, write, approval, export, merge, and rollback event
Retention	How long raw packets, derived packets, notes, and snapshots survive
Explanation	Why access was granted, why an item is visible, why an alert fired
Emergency controls	Break glass mode, dual control, elevated review, temporary restrictions

38. The login screen

The login screen needs to do more than look secure. It has to communicate session trust, environment state, and what mode the user is entering.

```

+-----+
| OMNI  |
| Secure Access Portal |

```

Organization / Tenant

Username

Credential method

Sign In

Local Node Sign In

Continuity Access

Session notes:

- Current environment status

- Last sync time for local authorization pack

- Device trust posture

- Available operating mode

A strong login screen should show: current environment, whether the node is online or degraded, whether local authorization material is current, what capabilities are reduced in offline mode, and a clearly audited break glass path for emergency continuity access.

39. Update, rollback, and tray utility

This is one of the best parts of your brief because it is practical. A tray utility is exactly the sort of detail that makes a real desktop product feel complete.

Tray utility functions

Tray option	Purpose
Open OMNI	Brings the main operator window forward
System state	Shows current operating mode, sync lag, storage pressure, and job status
Check for updates	Queries the release channel and downloads the staged package
Validate staged release	Runs package integrity and dependency checks before swap
Install update	Switches to the new release only after validation
Revert to previous snapshot	Rolls back to the last healthy release package
Export diagnostics	Builds a diagnostic bundle for later upload or review
Settings	Opens local node settings and release channel preferences

The golden rule here is simple. Never make update and rollback clever. Make them boring, visible, reversible, and logged.

Part VI. Analytics workflow, evidence handling, and operator trust

40. The issue development workflow inside the product

One of the best ideas in the tradecraft material is that many failures start before analysis starts, because the question itself is poorly framed. OMNI should therefore have an issue development board built into the case workspace.

9. Restate the issue in plain language.
10. State what is inside scope and outside scope.
11. Flip the issue and ask for the opposite framing.
12. Broaden the focus to adjacent drivers and dependencies.
13. Narrow the focus to the exact decision that must be informed.
14. List explicit assumptions currently being made.
15. List what evidence would most challenge those assumptions.

This sounds simple, but it changes the quality of the platform. It keeps a case from beginning as a fuzzy folder title and forces it to begin as a disciplined question.

41. Source check, quality check, and relevance check as first class product surfaces

The primers are very good on this point. Source reliability is not the same thing as information quality, and neither of those is the same thing as relevance. OMNI should keep those judgments separate because analysts mix them up when the app mixes them up.

The three review surfaces should never collapse into one rating

Review surface	What it answers
Source check	Who produced the information, what access they had, what their track record is, what collection or translation limitations exist
Quality check	How complete, ambiguous, corroborated, and internally consistent the information itself is
Relevance check	Why the item actually matters to the case question or the current hypothesis

A well designed evidence packet page should therefore contain three separate scoring areas, three separate comment areas, and a visible last reviewed timestamp.

42. Analysis of competing hypotheses inside OMNI

A platform like this should make it easy to build and maintain a hypothesis matrix. The user should be able to add hypotheses across the top, evidence down the side, and score each row as

supporting, neutral, or conflicting. Then the product can help surface which hypotheses remain plausible, where evidence is thin, and where hidden assumptions may be driving premature closure.

The product should not tell the analyst what to believe. It should expose the shape of the reasoning.

43. Indicators and signposts

Indicators are one of the most useful product features you can add because they connect analysis to time. A good case does not merely arrive at a conclusion. It also states what would change that conclusion.

So each case should support an indicators board with four simple columns: indicator, why it matters, what would count as confirmation, and what would count as contradiction. Tie those indicators to watchlists, regions, entities, or source streams where possible.

44. Alternative futures and scenario canvases

Some cases are not about what is true right now. They are about what could happen next. In those situations, a scenario canvas is better than a single summary paragraph. Let the team create a small set of plausible futures, list the assumptions behind each, and identify the earliest signs that each path may be unfolding.

This is especially important when the platform is being used to support planning rather than just retrospective understanding.

45. Operational environment analysis

Borrowing from the operational environment manuals, OMNI should support a structured environment workbook for serious cases. The workbook should ask the team to define the area and timeframe, list the significant environmental factors, describe how those factors affect friendly and external actors, identify the important actors, and capture plausible developments or branches.

Even when the case is not military in nature, this structured pass is useful. It makes people write down what terrain, infrastructure, weather, population patterns, traffic conditions, communications conditions, events, or institutional constraints matter.

46. Objective alignment and action package drafting

Here is the safe version of the planning loop. Every action package should answer six questions: what objective this package supports, what evidence justifies the package, what capabilities are assumed available, what approvals are required, what risks or unknowns remain, and what observable result would show that the package achieved its intended effect.

That gives the product a disciplined action layer without turning it into a black box automation machine.

47. Cases should be reviewable by another human

This sounds obvious, but it is surprisingly rare. A second analyst should be able to open a case and reconstruct what happened. That means OMNI needs a replay path.

- A chronological case timeline.
- A current state summary.
- A list of major findings with evidence backing.
- A list of unresolved contradictions.
- A list of open tasks and approvals.
- A last reviewed date for each core analytic surface.

If you skip replay, you lose team scale

The platform becomes dependent on individual memory. That is one of the fastest ways to make an expensive system underperform.

Part VII. Development roadmap, checks and balances, and team structure

48. Build order

The build order should follow the books, not the temptation of the flashiest demo. Start with the model and the case workflow. Then add evidence handling. Then add the map and timeline. Then add alerting. Then add analytics studio. Then add continuity and release management. Fancy AI and long tail integrations come later.

Recommended build sequence

Phase	Focus	Main deliverables
Phase 0	Naming, language, scope, architecture skeleton, design system, repo setup	Common glossary, context map, shell prototype, domain contracts
Phase 1	Case core and evidence core	Case workspace, evidence packets, source registry, audit baseline
Phase 2	Entity graph and map time workbench	Entity profile, map workspace, timeline, link explorer
Phase 3	Alerting and triage	Watchlists, rule engine, triage queue, compare view
Phase 4	Analytics studio	Issue framing, source check, quality check, relevance check, ACH, indicators
Phase 5	Workflow and approvals	Action packages, review routing, approval center, exports
Phase 6	Continuity mode and single node operation	Local node packaging, sync queues, storage pressure handling, break glass flows
Phase 7	Update and rollback system	Tray app, staged install, validation, snapshots, revert, diagnostics
Phase 8	Advanced adapters and specialized modules	Additional feeds, performance modules, optional model integrations

49. Checks and balances at every stage

You asked for checks and balances, and you were right to do so. A system this ambitious cannot rely on goodwill and memory alone.

Checks and balances

Review gate	What must be checked
Architecture review	Every new module must declare which bounded context owns it and which interfaces it consumes
Product review	Every screen must state the user question it answers and the domain object it exposes
Data review	Every new source family needs provenance, retention, validation, and confidence rules before full integration
Policy review	Every sensitive workflow needs an approval path and audit event model before release
Performance review	Native optimization is allowed only after measuring a real bottleneck
AI review	Any model assisted feature needs explanation behavior, fallback behavior, and human review rules
Continuity review	Every major feature must declare what it does in full mode, node mode, and continuity mode
Release review	No update becomes default without validation, health checks, and rollback readiness
Usability review	A second operator must be able to replay and understand a case without asking the original analyst

50. Squad structure

Do not organize the team around technical layers only. Organize around bounded contexts plus a small platform spine.

Suggested team split

Squad	Owns
Platform spine	Architecture, shared contracts, auth, policy, audit, sync, release tools
Case and workflow squad	Case workspace, tasks, approvals, action packages
Evidence and source squad	Source registry, packet normalization, provenance, review surfaces
Geo temporal squad	Maps, tracks, zones, playback, timeline, saved spatial views

Analytics squad	Hypothesis tools, indicators, scenario canvases, explanation surfaces
Reliability and continuity squad	Single node mode, storage pressure, diagnostics, rollback, recovery
Design system and operator experience	UI shell, interaction grammar, accessibility, keyboard flows, visual polish

51. Definition of done for a serious enterprise feature

- A named domain object or use case exists for the feature.
- The feature fits a bounded context cleanly.
- Happy path and failure path have both been demonstrated.
- Audit events exist for important state changes.
- Permissions and policy behavior are visible and tested.
- Continuity mode behavior is declared.
- UI copy is understandable without a meeting.
- Diagnostic output exists for support teams.
- Rollback impact is known if the feature ships in a release package.

52. Testing strategy

Testing needs to mirror the architecture. Inner layers should be extremely testable. Outer layers should be integration tested. Workflow surfaces should be scenario tested. The desktop shell should be smoke tested. Continuity mode should be chaos tested, or at least 'bad day' tested, not just unit tested.

Testing layers

Test layer	Examples
Domain tests	Invariants, merges, splits, confidence rules, workflow state transitions
Use case tests	Case creation, evidence review, alert promotion, action package drafting
Adapter tests	Storage, search, map, model, messaging, auth contracts
UI workflow tests	Open case, compare entities, review packet, approve package, rollback update
Continuity tests	Offline sign in, stale auth pack, sync backlog, storage pressure, reconnect
Release tests	Install, validate, switch, rollback, diagnostic export

Part VIII. Capability families kept visible at a deliberately high level

This part is here for one reason. You asked that major idea families remain present rather than silently removed. So they are here. But they are intentionally framed at the level of platform planning, boundaries, interfaces, approvals, and data model impact, not at the level of operational playbooks or intrusive step by step instructions.

53. Geospatial and movement intelligence features

At a high level, OMNI should support location based entity views, route overlays, time coupled playback, geofenced alerts, location confidence, area comparison, and map linked evidence packets. The platform should treat coordinates, regions, tracks, stops, dwell windows, and route hypotheses as first class geospatial concepts.

What this manual intentionally does not do is specify sensitive data acquisition procedures, covert collection tactics, or intrusive tracking mechanics.

54. Signal and spectrum related feature placeholders

Your preserved brief explicitly mentions many signal families. In planning terms, that means the platform should be able to accept externally produced signal derived observations through adapters, normalize them into evidence packets, assign provenance and confidence, map them in time and space, and link them into cases and watchlists.

At a platform level, that implies source family tags, quality metadata, confidence notes, location uncertainty handling, time window grouping, and analytic overlays. The detailed collection, exploitation, or offensive use of those signals is outside this handbook.

55. Camera, imagery, and video analytic features

At a high level, the product should support imagery and video packets with timestamps, locations, frame summaries, linked detections, human annotations, and replay from the evidence explorer and map workspace. The key design choice is that video should be treated as evidence with provenance and review state, not as a media attachment floating outside the case model.

56. OSINT and public information workflows

OMNI should treat public information as a source family with its own provenance, translation notes, authorship data, corroboration state, and relevance review. Public reporting is often messy, duplicated, and tempting to overtrust when it arrives in high volume. The product should therefore make it easy to review, cluster, and compare public information without letting it dominate by sheer quantity.

57. Subject or person of interest style monitoring placeholders

The preserved brief clearly imagines monitoring functions. The safe and useful planning level treatment is this: the platform may support authorized case linked subject profiles, watch status, change indicators, evidence backed summaries, review dates, and approval checkpoints. Identity confidence, source traceability, and legal or policy gating become central here.

What this handbook does not provide is any invasive operational playbook, covert procedure, or hidden monitoring instruction.

58. Security assessment and authorized testing placeholders

The preserved brief also mentions security testing. At the product planning level, that means OMNI can reserve a bounded context for importing authorized assessment results, linking them to assets or environments, scoring severity, tracking remediation workflow, and attaching those results to cases or planning packages.

This manual does not describe exploit development, intrusion procedure, or unauthorized access methods.

59. Action planning and coordination placeholders

The platform can still support action planning at a lawful planning level. That means reviewable action packages, objective alignment, assumptions, required approvals, risks, dependencies, time windows, and post action assessment criteria. Keep the planning object. Keep the review path. Keep the audit. Do not hide the reasoning.

60. Why retaining these categories still helps even at a vague level

Keeping these categories visible matters because it protects the long term architecture. If you pretend a capability family does not exist during planning, you will design the core model too narrowly and then bolt the category on later in awkward ways. It is better to leave a bounded context seam, a contract slot, and a policy gate now than to fake simplicity and pay for it later.

Appendix A. Preserved brief, verbatim

The text below is preserved exactly so the full original brief stays attached to the planning book. This appendix is here to preserve scope memory. It does not imply that every capability described below is implemented in this handbook at an operational detail level.

then based off that file which were going to keep and save for later, lets make a second massive, long, comprehensive in every way, text file/book/manual but this one is focused on using the skills we learned in the books to make a palantir look alike app that has a god view level of control, view and actions, like gotham and other palantir apps that take tons of data and turn it into useable actions.

You should act as a initial planner of a product/app developer, also with input from the product designer, a specialist in app making and design.

We want to make a palantir look alike app visually and functionally that has a god view level of control, view and actions, like gotham and other palantir apps that take tons of data and turn it into useable actions, the main focus will be on making it a palantir clone but you can use bilawa sidhus app worldview that he made also as a reference of lesser authority.

The users of this app will be military and government personel, meaning they will be taking tons of data from every sector, image data, gps data, signals (wifi, bluetooth, ssid, lte and all other types of signals) osint, camera data, traffic data, literally every type of data imaginable and this app will process it and make the palantir type of app (visually and functionally identical) described earlier which will turn that data into useable actions.

The output will follow the structure given from the books that we learned and will be made into a windows app that can be downloaded and used by military and government agencys such as the airforce, or any other military branch, intelligence agencies like nsa, fbi, and CIA. this native windows app will have tools relevant for all of those branches, and it will be able to do this because it has such a large and diverse pool of data and access to data.

the purpose of this app will be to use the massive amount of data and intuitively use it to provide targets, generate plans of action, such as surveillance, pentesting, tracking, alerts, and others that will be intuitively created based on the target and the data, and implement those actions based on the user input. it will also be used to monitor personal of interest (the CIA, NSA and FBI will use a lot of this function)

This apps primary platform will be on windows as a native windows app built with c#, c++, and all other languages needed for it to fully complete its task. The limitation will be that this app must be able to function on any windows or Linux computer meaning, that the data will not be stored directly on board the device for the most part, rather the app will grab information from the internet and process it using artificial intelligence.

you will also create a course of plan that breaks the creation of this app into smaller bits attainable by the developer team. you will flesh out the full structure of the app using the skills and knowledge from the books, your knowledge of Palantir and how they did it, and also how bilawa sidhu did it.

the app will once downloaded have a secure login required before the actual app can be accessed. this login screen which protects access to this state secret level app will work fully and be able to function in a doomsday scenario when all servers are down and a laptop or a single separate pc tower is forced to be made into a makeshift server.

speaking of services being down, this app though it will periodically check for updates from a GitHub repo, will be able to function even when the servers are down, since it is a native windows app and has the tools to analyze the data built into it and natively stored on the device and it grabs the data from the internet making it

have the capability to function without servers being online. there will also be a tray icon in windows bottom menu which has options like settings of the app and "check for updates where it will check for a new release of the app and once it has fully downloaded the new release, and it knows the code is functioning and that is isn't missing anything it will implement the update and install it. there will also be a revert option in the icon tray next to the check for updates to revert it to the older version in case a bad update goes through. this will revert it to one of the saved older versions (like a snapshot) and will also provide error diagnostics on what errors happened with the new update and what went wrong, this will be forwarded to the server so it can have knowledge of the errors in the update.

the app will be named omni and will be owned by a parent company invincible.inc

each step of the way in the development plan make sure that there is checks and balances.

break up the work into attainable sections that can be made independently before being integrated into the windows app. that way these smaller bits can be tested if they work and then can be added and then the whole app with the new section addition can be tested for functionality.

remember this is a military/government grade data analytics and actions tool but its also made to be able to run on a computer in the case of three scenarios, 1. all is well and there are servers and machines running the backend. 2. a severe outage where there is only one pc tower or laptop running with a powerful gpu, 4tb of storage max, and a decent cpu running the backed. and 3. a doomsday event where the program is required to run off of just the one computers hardware and use only data publicly available on the internet, osint, or data easily broken into to complete the same data analytics and actions available in Palantir's Gotham, (and only for the sake of more feature ideas and features for the app, bilawa sidhus worldview app.)

Appendix B. Working glossary for the OMNI team

Glossary

Term	Definition
Adapter	A translator at the system edge that converts between OMNI contracts and external tools or services.
Approval package	A structured, reviewable bundle that explains a proposed next step and its evidence basis.
Audit event	A durable record of a meaningful action, state change, or access event.
Bounded context	A clearly owned slice of the model with its own rules and language boundary.
Case replay	A reconstructed view of how a case changed over time and why.
Confidence posture	The current overall confidence level attached to a finding, profile, or case state.
Continuity mode	A degraded operating mode designed to preserve core

	work during major outages.
Derived view	A map, graph, table, or timeline projection built from core packets and entities.
Evidence packet	The reviewable unit of incoming or normalized information used by analysts.
Indicator set	A maintained list of signs that could strengthen, weaken, or alter a case judgment.
Local authorization pack	A locally stored package that allows constrained access when central services are unavailable.
Projection	A read optimized representation of core data for a particular workspace.
Snapshot	A frozen version of a case or release state kept for replay or rollback.
Workspace	A saved arrangement of panes, filters, and selections tailored to a user task.

Appendix C. Detailed design notes for continuity mode

This appendix goes deeper on one of the most important practical issues in the brief: keeping the product useful when the world is not normal.

C1. Local node philosophy

A local node should not pretend to be the whole enterprise. It should behave like a disciplined local subset. That means it should know what data it has, what data it does not have, what permissions have been cached, what models are available locally, how much storage remains, and which queued changes still need to be synchronized later.

- Keep a local catalog of cases pinned for continuity use.
- Keep compact evidence indexes for those cases.
- Keep a readable sync backlog with age and size counters.
- Keep a local health panel for storage, models, queue depth, and integrity.
- Keep a visible mode banner on every screen so no one mistakes local state for global completeness.

C2. Sync conflict rules

The app should not silently overwrite case state during reconnection. Draft notes, evidence annotations, task assignments, and approval states each need a conflict strategy. Some can merge automatically, some need human review, and some should never be auto merged.

Continuity sync conflict handling

Object type	Suggested conflict rule
Free text notes	Version and merge, with clear conflict markers if overlap exists
Structured tags	Union where safe, manual review where tags carry exclusive meaning
Assignments	Latest approved state wins, but all prior changes remain visible in audit
Approval decisions	Never auto merge contradictory outcomes, force review
Entity merges or splits	Never auto merge if continuity node and central node diverged materially
Watchlist rules	Queue for review if both sides changed threshold logic

C3. Break glass logic

If a break glass mode exists, it must be narrow, visible, timed, and heavily logged. The operator should know when they are in it. Supervisors should be able to review it later. The product should ask for a reason, mark the session, and restrict the resulting capability set to the smallest useful emergency subset.

Appendix D. Sample daily workflows**D1. Watch officer flow**

16. Open the command overview.
17. Review critical alerts and compare new alerts with the prior shift.
18. Promote the important alert into an existing case or create a new case shell.
19. Open the map workspace and verify that time window and region are correct.
20. Check the evidence packet quality and source notes before escalation.
21. Assign the case lead and add a summary note for handoff.

D2. Analyst flow

22. Open a case and restate the issue in the issue development board.
23. Review the current evidence packets and complete source, quality, and relevance checks.
24. Update the hypothesis matrix.
25. Refresh the indicators board and mark what changed since the last review.
26. Write or revise findings with citations to evidence packets.
27. Route the case for peer review or approval when ready.

D3. Platform administrator flow

28. Check operating mode distribution across nodes.
29. Review sync backlog, storage pressure, update status, and failed jobs.
30. Inspect policy violations, export events, and break glass sessions.
31. Stage the next release for validation if needed.
32. Export diagnostic bundles from any unhealthy nodes.
33. Approve rollback if the new release fails validation or operator acceptance.

Appendix E. What not to compromise on

When a project gets this ambitious, teams start trading away the wrong things for speed. Do not trade away these items.

- Do not trade away provenance.
- Do not trade away replayability.
- Do not trade away bounded context ownership.
- Do not trade away visible degraded state.
- Do not trade away reversible updates.
- Do not trade away human readable explanations.
- Do not trade away audit trails.
- Do not trade away grammatically clear UI copy and documentation.